

VectorLock

Backend & Engine Documentation

Architecture: Node.js (V8) + Native C Shared Library (FFI)

1. System Overview

VectorLock backend operates as a high-performance logistics decision engine designed for dynamic urban environments. The system calculates optimal routing for a fleet of delivery trucks across a predefined 25x25 grid, dynamically handling map obstacles, traffic congestion, and strict payload capacity constraints.

To achieve high-frequency pathfinding without blocking the Node.js event loop, the system employs a two-tier architecture:

- **Logistics Engine (C):** A highly optimized compiled shared library (.so / .dll) responsible for CPU-heavy computations, including A* pathfinding, spatial heuristic evaluations, and load factor calculations.
- **Application Server (Node.js):** A real-time Socket.IO server that manages client WebSocket connections, coordinates the simulation tick-rate, generates dynamic traffic phenomena, and bridges calls to the C engine via a Foreign Function Interface (FFI).

Project Scope (MVP / Proof of Concept): The current iteration of VectorLock is an MVP (Minimum Viable Product) and a strict Proof of Concept. The complete, long-term architecture for VectorLock includes a dedicated driver-side mobile application to sync real-world fleet coordinates with the main engine. However, as testing with physical moving trucks is not currently feasible, this build focuses entirely on the core logistics engine and simulated environment. This PoC is designed to explicitly demonstrate that this high-frequency, dynamic routing model is mathematically possible, highly scalable, and a game-changing approach to urban logistics.

2. Complete Repository File Manifest

Node.js Backend & API (Root)

- server.js - Core WebSocket server and tick-based simulation state machine.
- index.js - The Foreign Function Interface (FFI) Koffi abstraction wrapper.
- package.json / package-lock.json - Defines NPM dependencies and deterministically locks the dependency tree.

Test Suite

- ffi.test.js - Unit tests validating the C Engine memory safety without Node.js web overhead.
- test_koffi.js - Minimal sanity check for Koffi struct allocation.
- test-socket.js - End-to-end headless client integration test validating event emissions.

C Logistics Engine (/engine/src)

- main.c - Standalone CLI simulator for testing C logic independently.
- mapping.c / mapping.h - Implementation of the 25x25 map grid, hardcoded static routes, and A* pathfinding.
- functionSpecs.c / functionSpecs.h - Load calculation constraints and dynamic dispatch decision logic.
- DeliveryRoutes.h - Centralized memory structures for truck and shipment states.

Build System & Artifacts (/engine)

- Makefile - Build instructions for compiling the C shared library.
- build/libengine.so / build/engine.dll - The final compiled library consumed by Node.js.

3. Core Data Structures & Memory Optimizations

To allow zero-copy memory sharing and prevent serialization overhead between JavaScript and C, data structures are explicitly mapped using `koffi.struct`.

- **Memory Optimization:** To minimize the memory footprint during FFI serialization, `row`, `col`, and `routeSymbol` are strictly cast as 8-bit integers (`int8` / `int8_t`) rather than standard 32-bit integers.

Struct	Description	Properties & Limits
Point	2D coordinate on the grid.	<code>row (int8_t)</code> , <code>col (int8_t)</code> .
Route	Path representation.	<code>points (Point[100])</code> , <code>numPoints (int)</code> , <code>routeSymbol (int8_t)</code> . Limit: Hardcapped at <code>MAX_ROUTE</code> (100 points) to prevent buffer overflows.
MapStruct	25x25 grid layout.	<code>squares (int[25][25])</code> , <code>numRows (int)</code> , <code>numCols (int)</code> .
ShipmentInfo	Payload requirements.	<code>weight (int)</code> , <code>volume (int)</code> , <code>destination (Point)</code> .
TruckInfo	Truck state & constraints.	<code>routeSymbol (int8_t)</code> , <code>maxWeight (int)</code> , <code>maxVolume (int)</code> , <code>currentWeight (int)</code> , <code>currentVolume (int)</code> , <code>route (Route)</code> , <code>currentLocation (Point)</code> .

4. The C Logistics Engine (/engine)

4.1 Fleet Specifications & Symbols

The engine maintains limits for a three-truck fleet. Hardcoded values enforce realism across routing logic:

- Global Capacity Limits: Maximum Weight: 2000kg, Maximum Volume: 20 units.
- Blue Line: Assigned route symbol 2.
- Green Line: Assigned route symbol 4.
- Yellow Line: Assigned route symbol 8.
- Dynamic Diversions: Assigned route symbol 16 (DIVERSION).

4.2 A* Pathfinding Mechanics (mapping.c)

The `shortestPath` function implements a highly optimized grid-based A* algorithm tailored for vehicular movement, avoiding floating-point math entirely to maximize CPU cache performance.

- Octile Heuristic (Integer Math): The `calculateHCost` function utilizes exact integer multipliers (14 for diagonal, 10 for orthogonal) rather than standard Euclidean floats.
- Anti-Zig-Zag Penalty: To ensure realistic truck movement, the algorithm tracks the current vector. If a proposed move requires changing direction, a strict Turn Penalty of 10 is added to the node's tentative G-cost, heavily favoring straight lines.
- Collision Rules: It strictly prevents "corner-cutting" diagonally through a gap between two adjacent buildings/obstacles.

4.3 Dispatch Algorithm (functionSpecs.c)

The `findTruckForShipment` function routes shipments by evaluating constraints sequentially:

- Capacity Check: Rejects the assignment if `currentWeight + shipment.weight` or `currentVolume + shipment.volume` exceeds the vehicle limits.
- Distance Evaluation: Evaluates the dynamic A* distance from the truck's exact current location to the destination.
- Tie-Breaker: If two valid trucks share the same distance, the engine calculates a relative `LoadFactor` and assigns the shipment to the truck with the lowest utilization ratio.

5. The Node.js Application Server & FFI

5.1 FFI Abstraction & Graceful Degradation (index.js)

The index.js file handles the cross-platform Foreign Function Interface to libengine.so or engine.dll depending on os.platform(). If the shared library fails to load, a try/catch block suppresses the fatal exception and injects JavaScript mock functions (e.g., returning empty static arrays for routes) ensuring the frontend and Node UI can still boot.

5.2 Tick-Based Simulation & Fail-Safes (server.js)

The WebSocket server runs state machines on fixed intervals:

- `manageTruckLoop()`: Ticks every 500ms by default, but accelerates to 150ms when the mode is swarm and density is Critical to handle rapid path recalculations. It processes waypoint queues, advances truck coordinates, and verifies path validity against dynamic obstacles.
- Infinite Recalculation Fallback: If a truck's path is severed by a dynamic obstacle and it has not yet reached its destination, the state machine enters a loop. It continuously attempts to call `calculateDiversionPath` on every tick until a valid path opens up.
- `manageTrafficLoop()`: Periodically generates pseudo-random grid congestion based on configured densities.
- Zero-GC BFS Optimization: To prevent Garbage Collection (GC) pauses during high-frequency ticks, the `isSolvable` validation uses flat, pre-allocated 625-byte TypedArrays (Uint8Array for visited maps, Int8Array for row/col queues) instead of standard JavaScript arrays.
- Strict Spawning Protection (`isRestrictedCell`): To ensure trucks are never instantly paralyzed by spontaneous traffic generation, dynamic obstacles are strictly forbidden from spawning on:
 - The origin point [0,0].
 - The exact coordinates of any active truck.
 - Any truck's active `targetDestination`.
 - Any coordinates present in a truck's upcoming `waypointQueue`.
- 100-Attempt Limit: When generating random traffic, the engine attempts to find a solvable configuration a maximum of 100 times. If the randomized traffic chokes the

path after 100 attempts, it safely aborts and spawns [] (zero traffic) rather than trapping the server.

5.3 Routing Modes & Traffic Generation

- vectorLock (Static Dispatch): Manifests provide predefined waypoints. Trucks handle items in queue order. Traffic generates entirely new static road blocks exactly every 5 seconds (5000ms), forcing trucks to dynamically recalculate paths around the new configuration.
- swarm (Algorithmic Dispatch): Manifests set a target destination. Trucks abandon fixed queues to continuously calculate optimal A* routes converging dynamically on the target.
- Traffic Densities: The traffic generation algorithm scales the number of blocked grid cells based on specific percentages of available space: Low (1%), Mid (3%), High (7%), and Critical (22%).
- Swarm Migration Ratios: In swarm mode, traffic blocks physically migrate to adjacent cells. The swarm shifting interval speeds up from 1500ms to 150ms in Critical mode. In Critical density, 95% of active traffic blocks attempt to shift to a neighboring cell per tick, whereas in lower densities, only 20% of the traffic mass is permitted to move.

6. WebSocket API (Port 3001)

Clients connect via Socket.IO for real-time visualization and interaction.

Incoming Client Events

- `set_traffic_density` (payload: 'Low' | 'Mid' | 'High' | 'Critical'): Alters congestion density.
- `set_mode` (payload: 'vectorLock' | 'swarm'): Changes simulation behavior.
- `clear_traffic` (payload: { row, col }): Allows UI interaction to manually clear a roadblock.
- `reset_scene`: Resets the fleet, manifests, and map to default state.
- `dispatch_manifest` (payload: { mode, truckRoutes | manifest }): Triggers a dispatch.

Outgoing Server Events

- `init_map`: Transmits static building layouts.
- `traffic_update`: Broadcasts active dynamic traffic coordinates.
- `truck_positions`: Fired per-tick. Provides real-time physical fleet placement and paths.
- `fleet_status`: Broadcasts load weights and capacity usage.
- `waypoint_reached`: Emitted upon successful delivery.

7. Testing & Diagnostics

7.1 Standalone CLI Simulator (main.c)

The C engine acts as a standalone terminal application, skipping the JS layer for low-level validation.

- It loads the hardcoded map and prints an ASCII grid representation.
- It awaits standard input formatted as [Weight] [Volume] [Row] [Column] (e.g., 50 2 13 M).
- Accepts 0 0 x to gracefully terminate the REPL loop.

7.2 Integration Tests

- test_koffi.js verifies memory encoding/decoding of native C structs.
- ffi.test.js exercises the Node-to-C pipeline without Socket.IO overhead.
- test-socket.js acts as a headless integration suite, emitting a WebSocket dispatch payload and monitoring the event bus, waiting for exactly 5 movement ticks before concluding the test to ensure the engine isn't stalled.

8. Build System & Dependencies

8.1 Compiling the C Engine

The Makefile inside /engine builds the shared library required by the backend.

- Optimization: Uses the -O3 flag for maximum pathfinding performance.
- PIC Compilation: Enforces -fPIC (Position Independent Code) which is mandatory for creating .so shared libraries for dynamic loading.
- Commands: Run make to compile. Run make clean to scrub .o cache files.

8.2 Node.js Dependencies

Specified in package.json:

- socket.io / socket.io-client (^4.8.3): Bidirectional WebSocket capabilities.
- koffi (^3.0.2): Fast C FFI interface.

9. Deployment & Environment Requirements

Simulation Mode: Because this is a Proof of Concept, booting the backend currently defaults to a simulated local WebSocket environment. No external driver-side API keys or GPS hardware connections are required to run the server.js state machine.

To successfully build and deploy the VectorLock backend, the host environment must meet the following strict requirements:

- Node.js Runtime: Node.js version 18.0.0 or higher is strictly required for modern V8 Foreign Function Interface (FFI) memory bridging via Koffi.
- C Compiler Toolchain: The host machine must have a C compiler installed to build the native engine artifact before booting the Node server.
 - Linux / macOS: GCC or Clang, along with GNU make.
 - Windows: MSVC, MinGW, or Cygwin to correctly output .dll files.
 - *Note: The above two are usable, but WSL is recommended.*
- Cross-Platform Execution: The system automatically detects the OS during runtime (os.platform()) to load .so or .dll files. If the binary is missing or fails compilation, the backend gracefully downgrades to Javascript mock responses.

Terminal Execution Guide

1. Compile the C Engine

Always do this first if you change any C code, otherwise your backend will fall back to the dummy JavaScript functions.

Bash

wsl

cd path/to/engine

make clean

make

2. Run the Isolated Unit Tests (Server OFF)

Run these to guarantee your C logic and memory pointers are flawless before you boot the main app.

Bash

wsl

cd path/to/backend

node test_koffi.js

cd path/to/backend/tests

node ffi.test.js

(Once these finish, they automatically close, freeing up your memory.)

3. Start the VectorLock Server

Now that the engine is verified, start your actual application.

Bash

wsl

cd path/to/backend

node server.js

(Leave this running! Your UI/frontend can now connect to port 3001.)

4. Run the Integration Test (Optional)

If you ever do want to run test-socket.js to verify the trucks are moving, you must open a second WSL terminal window while Step 3 is still running and execute:

Bash

cd path/to/backend

node test-socket.js

(Once the run is over, refresh the web page to avoid conflict!)

Frontend UI & Visualization Documentation

1. System Overview

The VectorLock frontend is a real-time, 3D visualization dashboard designed to monitor and control a logistics dispatch engine. Built on Next.js, it utilizes React Three Fiber (R3F) to render a live 25x25 grid map containing buildings, dynamic traffic, and fleet movements. It maintains a persistent WebSocket connection to the backend to ensure zero-latency UI updates, fleet tracking, and algorithmic routing synchronization. For the scope of this Proof of Concept, this 3D visualization dashboard serves as a functional stand-in for physical fleet tracking and the future driver-side application. It allows us to monitor dispatch payloads, algorithmic routing, and real-time engine telemetry in a simulated environment without requiring physical hardware on the road.

2. Technology Stack

- Framework: Next.js (App Router) with React 18+.
- 3D Engine: Three.js wrapped in @react-three/fiber and @react-three/drei for declarative WebGL scenes.
- Styling: Tailwind CSS for layout, animations, and utility-class styling.
- Real-time Comms: socket.io-client.
- Icons: lucide-react.
- Fonts: Vercel's Geist and Geist_Mono.
- Native APIs: Utilizes the native Web Crypto API (crypto.randomUUID()) to generate unique IDs for manifest items.

3. Client-Side Algorithms (optimizationPipeline.ts)

Before the frontend dispatches a payload to the backend C Engine, it processes the raw user manifest through an internal optimization pipeline to minimize server-side computational load. The optimizeDispatch function executes a two-step algorithmic pass:

1. Strict Bin Packing (First-Fit): It groups items based on the fleet's physical constraints (TRUCK_MAX_WEIGHT = 2000kg, TRUCK_MAX_VOLUME = 20m³). It completely maximizes a single truck's load before opening a new "bin". If the manifest exceeds the capacity of the 3 active trucks, the excess is pushed to an overflow queue array.
2. Empty Bin Stripping: Before sequencing, the pipeline intentionally strips out any empty bins (filter(trip => trip.length > 0)) to ensure the system never dispatches an empty "ghost" truck.
3. TSP Nearest Neighbor: For the items assigned to each truck, the pipeline sorts the waypoints sequentially to minimize grid travel distance using Manhattan distance calculations ($\text{Math.abs}(x1 - x2) + \text{Math.abs}(y1 - y2)$). It then automatically appends a Base (Return) coordinate at the end of every route.

4. Component Architecture

4.1 The Central Hub (Dashboard.tsx)

Dashboard.tsx acts as the single source of truth for the application state, mounting the Map, Command Panel, and Queue Monitor.

- **State Managed:** Tracks manifest, overflowQueue, activeWaypoints, optimizedRoutes, targetPos, mode, and trafficLevel.
- **Traffic Mode Failsafe:** Implements a critical failsafe state transition. When the user switches the routing mode from swarm back to vectorLock, the Dashboard automatically downgrades the trafficLevel from Critical to High and emits this to the server to prevent vectorLock from getting trapped in an unnavigable state.
- **Lifecycle:** Emits reset_scene when switching routing modes to ensure clean state transitions, and listens for the waypoint_reached socket event to remove completed deliveries from the active queue.

4.2 The 3D Engine (GridMap.tsx & Trucks.tsx)

The GridMap component handles all WebGL rendering within a <Canvas> wrapper.

- **Grid & Environment:** Renders a 25x25 grid with ambient and point lighting.
- **Interaction & Dynamic Preview Cursor:** Translates 3D pointer events into 2D grid coordinates. The map calculates a previewPos on handlePointerMove and renders a transparent ghost cylinder/cone under the user's cursor. This cursor dynamically turns red (isBlocked) if hovering over buildings/traffic, or teal if the placement is valid.
- **Traffic Override:** Prevents users from clicking on static buildings or active traffic blocks, *unless* the app is in swarm mode with Critical traffic, where it allows clicking a traffic block to manually clear it (socket.emit('clear_traffic')).
- **Animation & Interpolation:** Smoothly animates truck meshes between grid nodes. There are two variations of this implementation in the codebase:
 - The isolated src/components/Trucks.tsx module utilizes useFrame and THREE.MathUtils.lerp for smooth interpolation.
 - The actively mounted GridMap.tsx relies directly on Three.js's native Vector3.lerp() method (meshRef.current.position.lerp()) to update the target position vector.

4.3 The Control Center (CommandPanel.tsx)

An absolute-positioned UI overlay that handles user input and server dispatching.

- **Live Feed:** Dynamically renders progress bars calculating the current weight/volume percentages based on real-time server telemetry.
- **Input Validation & Error UI:** Prevents drafting items that exceed the 2000kg/20m³ limits in vectorLock mode. It manages temporary error and status string states that render animated toast-like messages (e.g., "Item added to manifest", "Limit Exceeded") which automatically clear via `setTimeout`.
- **Queue Processing Engine:** Housed inside a `useEffect` hooked into the `truck_positions` socket event. Every time the server sends truck data, it checks if a truck's server-side `isIdle` flag is true. If a truck is idle and the overflow queue has items, it automatically unshifts the next trip from the queue, zeroes out the truck's local payload state, and immediately emits a new `dispatch_manifest` event.

4.4 The Queue Monitor (QueueMonitor.tsx)

A read-only overlay on the right side of the screen displaying the status of logistics operations.

- Divides data into three visual tiers: Draft Dropoff (Pre-dispatch), Ongoing Trips (Active network), and Up Next (Overflow items waiting for available trucks).
- Allows deletion of draft items, which triggers an immediate recalculation of the `optimizeDispatch` pipeline.

5. WebSocket Integration (socket.ts)

The socket client is initialized as a singleton targeting `http://localhost:3001` (or a `NEXT_PUBLIC_API_URL` env variable) with `autoConnect: false` to allow the Dashboard to dictate the connection lifecycle.

Event	Direction	Description	Component Handling
init_map	Server → Client	Receives static building layouts for the grid.	GridMap.tsx
traffic_update	Server → Client	Receives dynamic congestion block coordinates.	GridMap.tsx
fleet_status	Server → Client	Receives payload capacity metrics (Current & Max Weight/Vol).	CommandPanel.tsx
truck_positions	Server → Client	Receives 3D coordinates, idle flags, and planned route arrays for the fleet.	GridMap.tsx, CommandPanel.tsx
waypoint_reached	Server → Client	Receives confirmation when a truck hits a node. Decrements the live weight/volume payload in the UI and removes the marker from the map.	CommandPanel.tsx, Dashboard.tsx
dispatch_success	Server → Client	Confirms a successful payload sent to the C Engine. Clears loading states and sets success UI messages.	CommandPanel.tsx

Event	Direction	Description	Component Handling
dispatch_manifest	Client → Server	Sends structured payload of routes/targets to the C Engine.	CommandPanel.tsx
set_traffic_density	Client → Server	Triggers whenever the user toggles the traffic density buttons (Low, Mid, High, Critical).	CommandPanel.tsx, Dashboard.tsx
set_mode	Client → Server	Syncs the current routing logic mode (vectorLock or swarm) with the backend engine.	Dashboard.tsx
reset_scene	Client → Server	Requests the backend engine to purge active routes, queues, and reset trucks to base when switching modes.	Dashboard.tsx
clear_traffic	Client → Server	Dispatched only in swarm mode with Critical traffic when a user explicitly clicks a traffic block to manually clear it.	GridMap.tsx

6. Complete File Manifest

Configuration & Entry Points

- `src/app/layout.tsx` - Root HTML wrapper, injects Geist fonts.
- `src/app/page.tsx` - Main entry route, mounts the `<Dashboard />` component.
- `src/app/globals.css` - Tailwind directives and base CSS resets.
- `src/app/favicon.ico` - Application icon.

React Components (`/src/components`)

- `Dashboard.tsx` - Main state controller and layout wrapper.
- `CommandPanel.tsx` - UI overlay for drafting manifests, setting modes, monitoring telemetry, and driving the automated queue dispatcher.
- `QueueMonitor.tsx` - UI overlay for viewing draft, active, and queued shipments.
- `GridMap.tsx` - The R3F WebGL canvas, rendering the grid, buildings, traffic, RoutePreview markers, and active truck meshes.
- `Trucks.tsx` - An isolated 3D truck component module utilizing `useFrame` for position interpolation and emissive pulsing (Kept as a modular asset, though core fleet rendering logic currently resides natively inside `GridMap.tsx`).

Libraries & Utilities (`/src/lib`)

- `optimizationPipeline.ts` - Client-side bin packing and TSP algorithms.
- `socket.ts` - Socket.IO client initialization.

7. Local Development & Execution Guide

To successfully run the VectorLock frontend, it must be executed in tandem with the Node.js/C-Engine backend. The frontend relies on a persistent WebSocket connection to render the 3D environment and will appear empty if the backend is offline.

1. Installation

Navigate to the dashboard project directory (where your Next.js configuration resides) and install the required dependencies (Next.js, Three.js, React Three Fiber, Socket.io-client, etc.):

Bash

```
cd path/to/dashboard
```

```
npm install
```

2. Start the Frontend UI

While the server.js is running open a new terminal window, navigate to the dashboard directory, and start the Next.js development server:

Bash

```
cd path/to/dashboard
```

```
npm run dev
```

VectorLock Operator's Manual

Dashboard Interface & Simulation Guide

1. Interface Overview

VectorLock is a real-time 3D logistics dashboard designed to calculate optimal delivery routes in dynamic environments. The interface is divided into three primary zones:

- The 3D Grid Map (Center): The live visualization of the 25x25 city grid, your delivery fleet, static buildings, and dynamic traffic congestion.
- The Command Panel (Left): Your primary control center for viewing live truck telemetry, changing routing modes, and drafting new shipments.
- The Queue Monitor (Right): A live breakdown of your logistics network, split into Draft Dropoffs (pre-dispatch), Ongoing Trips, and Up Next (overflow items queued for available trucks).

2. Camera Controls & Map Legend

You can freely explore the 3D environment using your mouse:

- Rotate Camera: Left-Click + Drag
- Pan Camera: Right-Click + Drag
- Zoom: Scroll Wheel

Map Visuals & Markers:

- Dark Grey Blocks: Static Buildings (Impassable).
- Red Blocks: Dynamic Traffic Congestion (Impassable).
- Neon Pink/Red Cylinders: Active delivery targets currently assigned to a truck.
- Faded/Ghost Cylinders: Queued delivery targets waiting for an available truck.
- Flat Cyan/Teal Circle: Your current mouse-click selection on the map.
- Dashed Multi-Colored Lines: Projected optimal routes calculated by the A* engine *before* dispatch.
- Solid Lines (Blue/Green/Yellow): The active, real-time path the trucks are currently driving along.

3. Operating Modes

The engine operates in two distinct behavioral modes, which you can toggle at the top of the Command Panel:

A. vectorLock (Static Dispatch)

This is the standard, real-world logistics mode. You manually build a manifest of packages (with specific weights, volumes, and destinations). The engine automatically groups them into the most efficient bins, sorts them by nearest-neighbor distance, and dispatches the fleet. Traffic generates and updates every 5 seconds, forcing trucks to dynamically reroute.

B. Algo Test (Swarm Simulation)

This mode tests the engine's high-speed recalculation capabilities. Instead of a queue of packages, you assign a single global target. All three trucks will abandon their standard queues and use continuous A* pathfinding to dynamically converge on that target while traffic rapidly shifts around them.

4. Traffic Control

Below the mode toggle, you can adjust the Traffic Density (Low, Mid, High). This dictates how much of the grid is choked by red traffic roadblocks.

Critical Swarm Override (God Mode): If you switch the mode to Algo Test, a hidden Critical traffic density button appears.

- Critical mode spawns massive amounts of rapidly shifting traffic (updating every 150ms).
- Manual Clearing: If a truck gets completely boxed in and trapped during Critical mode, you can Left-Click directly on a red traffic block on the 3D map to manually delete it and force a path open for your fleet.

5. How to Dispatch a Fleet (Step-by-Step)

To run a standard logistics operation, ensure your mode is set to vectorLock.

Step 1: Draft an Item

1. Set Target: Left-click any empty square on the 3D grid. A cyan highlight will appear, and the coordinates will register in the Command Panel's "Target" field.
2. Enter Details: Type in an "Item Name" (e.g., Medical Supplies, Tech Hardware).
3. Set Load: Enter the Weight (max 2000kg per truck) and Volume (max 20m³ per truck).

4. Add to Dropoff: Click the button to add it to your draft manifest. Notice the dashed route preview lines appear on the map.

Step 2: Review the Queue Monitor

- Look at the right side of your screen. Your drafted items will appear under Draft Dropoff.
- You can repeat Step 1 to add as many items as you want. The frontend will automatically group them into the most efficient truck loads.
- If you make a mistake, hover over the item in the Queue Monitor and click the trash can icon to remove it.

Step 3: Dispatch

- Click the large white Dispatch button at the bottom of the Command Panel.
- The engine will assign the items to the available Blue, Green, and Yellow trucks. The neon markers will lock into the map, and the trucks will begin moving along their solid active paths.

6. Live Telemetry & Queue Management

Once trucks are dispatched, the system handles operations entirely autonomously.

- Live Data Feed: The top of the Command Panel displays live progress bars for the Blue, Green, and Yellow trucks. You can watch their Weight (WGT) and Volume (VOL) capacities fill up upon dispatch, and instantly drop when they reach a delivery target.
- Overflow Handling: If your manifest contained more items than the 3 trucks could carry at once (e.g., exceeding 6,000kg total weight or 60m³ total volume), the excess items will sit in the Up Next (Queued) section of the Queue Monitor.
- Auto-Reload: As soon as a truck finishes its current route and returns to base, it becomes "Idle." It will automatically pull the next available items from the Queue, calculate a new optimal route, and deploy again without any manual intervention.